Complexity and approximation solutions

A11-R01 Encoding and pseudopolynomial time

Binary length $N=m+1$, so time is $\Theta(t)=\Theta(2^{(N-1)})$, exponential in N. Unary length is t, making the loop linear in that encoding length. This does not make the binary version polynomial. An instance $\langle G,k\rangle$ includes graph bits, the bits of k, and separators. Vertex count alone can omit a long encoding of k.

A11-R02 A quantifier gap in a verifier

For the first, take three isolated vertices with $k=3$ and certificate $[1,2,3]$. It passes without a 3-clique. For the second, take path $1-2-3$ with $k=3$ and the same certificate. Edge $\{1,3\}$ is absent. Completeness is $\forall x\in L\ \exists c, |c|\leq p(|x|), V(x,c)=1$. Soundness is $\forall x\notin L\ \forall c, |c|\leq p(|x|), V(x,c)=0$. Also give a fixed polynomial halting bound.

A11-R03 Build a verifier from a decider

Let D be a polynomial-time decider for $L\in P$. Use the empty certificate ε with $p(N)=0$. Set $V(x,\varepsilon)=D(x)$ and reject other certificates. If $x\in L$, ε is accepted, giving completeness. If $x\notin L$, D rejects and no other certificate is permitted, giving soundness. Time inherits D's polynomial bound. This proves only $P\subseteq NP$, not equality or strict containment.

A11-R04 Audit reduction directions

(a) Sufficient. Every $L\in NP$ satisfies $L\leq pA\leq pB$, so B is NP-hard; use $B\in NP$. (b) Insufficient. For example $A=3SAT$ and $B=PATH$. A polynomial PATH decider can map yes/no to fixed satisfiable/unsatisfiable 3SAT formulas. This direction gives no hardness proof for PATH. (c) Still sufficient under the stated $B\in NP$ assumption; the exponential algorithm adds nothing needed. If $B\in P$, then $A\in P$ and NP-completeness of A gives $P=NP$.

A11-R05 Prove composition time

$h(x)=g(f(x))$. We have $x\in A$ iff $f(x)\in B$ iff $h(x)\in C$. Since f outputs $O(N^2)$ bits, total time is $O(N^2+(N^2)^3)=O(N^6)$. If each call doubles the current string, each call is linear in its current length but after N calls the length is $N\cdot 2^N$. Bound intermediate sizes and all calls in terms of the original input length.

A11-R06 Build the full 3SAT→CLIQUE graph

There are 9 occurrence vertices and k=3. E={15,16,24,26,34,35,36,17,18,19,27,29,37,38,48,49,57,58,59,67,68}, giving 21 edges. x=y=z=true satisfies F and {1,5,7} is a clique. Vertex 4 is ¬x and 7 is x, so they conflict. Vertices 6 and 7 are z and x in different groups, so they are adjacent. Forward: choose one true literal per clause; each pair is consistent, hence adjacent. Reverse: an m-clique has exactly one vertex per group; no selected literals conflict, so make them all true and extend arbitrarily to satisfy every clause.

A11-R07 Repair constructions with small counterexamples

Use F=(x∨x∨x)∧(¬x∨¬x∨¬x), with m=2, for both bugs. It is unsatisfiable. Under (a), two x occurrences in the first group form a spurious 2-clique. Under (b), x in one group and ¬x in the other form one. The correct graph has no cross-group edges and no 2-clique. An empty conjunction is true and maps to the empty graph with k=0; if zero is forbidden, use K_1,k=1. A clause containing x and ¬x still has no internal edges and needs only one true occurrence.

A11-R08 Recover a maximum clique from decision

Binary-search the monotone predicate D(G,k) on [0,n] to obtain k*=ω(G) in O(log(n+1)) calls. Fix k* and try each vertex v, deleting it if D(G−v,k*) says yes, with at most n further calls. Every step preserves a k*-clique. If more than k* vertices remained, a vertex outside a chosen k*-clique would be deletable, a contradiction. A previously indispensable vertex stays indispensable because deleting more vertices cannot create a clique. The remaining k* vertices form a maximum clique. If D takes O(L^d), subgraph encodings have length O(N), so total time is O((n+log(n+1))N^d+poly(N)). This uses multiple, potentially adaptive queries instead of producing a single many-one target instance.

A11-Z01 Solution

There are $T(n) = n^2 + 1 = O(n^2)$ transition layers plus an initial configuration layer, with n free witness bits. Each layer uses $O(s(n)) = O(n^2)$ gates, so the total is $O(Ts) = O(n^4)$. Hardwiring x specifies the instance while leaving every bit of y free; satisfiability is therefore equivalent to the existence of an accepting witness.

A11-Z02 Solution

$E[X] = 100 \times 7/8 = 87.5$. If the success probability is q, then $87.5 \le 100q + 75(1-q)$, hence $q \ge 1/2$. Ten independent assignments all fail with probability at most $(1/2)^{10} = 1/1024$. Expectation adds the clause indicators without requiring clause events within a trial to be independent.

A11-Z03 Solution

Given a nondeterministic machine M, construct a verifier. Its transition table is fixed and finite, so encode each branching choice with a fixed number of bits. At most $T(n)$ steps require an $O(T(n))$-bit witness, where $T(n)$ is polynomial. Starting from the initial configuration, the verifier reads choices, checks each is legal, and simulates the transition. On early halting it ignores remaining padding; an invalid encoding or illegal transition is rejected. An accepting branch supplies an accepting witness, and a legal witness accepted by the simulation specifies an actual accepting branch.

Conversely, given verifier V, a nondeterministic machine guesses each bit of a polynomial-length witness and then runs $V(x, y)$ deterministically, accepting exactly when V does. Guessing plus verification takes polynomial time on every branch. For variable-length witnesses, encode a length field and padding and validate the encoding. Thus an accepting branch exists exactly when an accepting witness exists.

A11-X01 Gate encoding

The clauses are (¬z∨a), (¬z∨b), (z∨¬a∨¬b). With (a,b)=(1,0), z=0 gives (1,1,1). If z=1, (¬z∨b)=0. More generally the first two clauses enforce z→a∧b and the third enforces a∧b→z, so their conjunction is equivalent to the gate.

A11-X02 Complement and cover

The complement has edges 13,14,24. Choose S={1,2}, a clique in G. Its complement C={3,4} touches 13,14,24, so C is a cover of size 2. In general S is a clique in G iff S is independent in the complement iff V−S covers every complement edge. Any clique of size at least k contains one of size k, and a cover of at most n−k can be enlarged to that size. Constructing all n(n−1)/2 possible pairs takes polynomial time.

A11-X03 Hamiltonian cycle to TSP

Each original edge receives weight 1; both diagonals receive 2. With B=4 the tour 1,2,3,4,1 costs 4. After deleting 41, the source is a path and has no cycle. Every target tour has four edges of weight at least 1; reaching cost at most 4 requires all four to be original edges, which would be a source Hamiltonian cycle. Thus none reaches the threshold. The complete graph has $O(n^2)$ edges and is constructed in polynomial time.

A11-X04 Decimal encoding

The selected variable numbers are 10010, 01011, and 00101, summing to 11122. Select slack 00020 and 00002 to reach target 11144. In any subset the variable columns total at most 2 and each clause column at most 6, so no column carries. Reaching the target forces exactly one of T_i,F_i for each i. Each clause needs digit 4, but its slack can supply at most 3; at least one selected literal number contributes 1, and that literal is true under the decoded assignment. Thus every clause is satisfied.

A11-X05 Edge gadget and cover

For x=0,y=1 the selected full path is a1,a2,a3,b1,b2,b3,b4,b5,b6,a4,a5,a6. Its external ports are a1,a6; each internal vertex has degree two on this path. When x=y=0, the forced edges instead form two disjoint six-cycles; their vertices already have cycle degree two and cannot connect to any selector. They cannot be part of a single Hamiltonian cycle spanning the larger graph. In the source path u−v−w, the example selects {v}, covering both edges; the one selector links the two incident v-sides into the 25-vertex cycle.

A11-A01 Directions and values

For minimization A=18, OPT=12 and maximization A=12, OPT=18, compute ratios and guarantees. Why is $A \leq \rho OPT$ insufficient for maximization?

For minimization, A/OPT=18/12=3/2, so $A \leq (3/2)OPT$, namely $18 \leq 18$. For maximization, OPT/A=18/12=3/2, so $A \geq OPT/(3/2)$, namely $12 \geq 12$. The maximization inequality $A \leq \rho OPT$ follows from feasibility $A \leq OPT$ and $\rho \geq 1$, so it does not ensure A is close to OPT.

A11-A02 Cover rounds and a tight bound

For E={ab,bc,cd,de,ef}, select ab, cd, ef. List residual edges, C, and a lower bound. Prove the ratio is exactly 2.

Use E={ab,bc,cd,de,ef} and choose ab, cd, ef in this order.

Round	Edge	Added vertices	Remaining edges deleted	Edges left
1	ab	a,b	ab,bc	cd,de,ef
2	cd	c,d	cd,de	ef
3	ef	e,f	ef	∅

M={ab,cd,ef} is a matching and C contains all six vertices. The cover {b,d,f} has size 3; the three disjoint edges force any cover to use at least 3 vertices. Thus OPT=3 and A/OPT=6/3=2, attaining the guarantee.

A11-A03 Maximal versus maximum

On a-b-c-d select bc. Prove M is maximal but not maximum, and refute taking one arbitrary endpoint per matching edge.

Let M contain the selected edges. Selecting uv deletes every edge incident to u or v, so future selected edges share neither endpoint. Thus M is a matching.

Any cover C* must cover each edge of M. These edges have disjoint endpoints, so they require distinct cover vertices. Hence $|C^*| \geq |M|$. The algorithm takes exactly two new endpoints per selected edge, giving $|C| = 2|M| \leq 2|C^*|$.

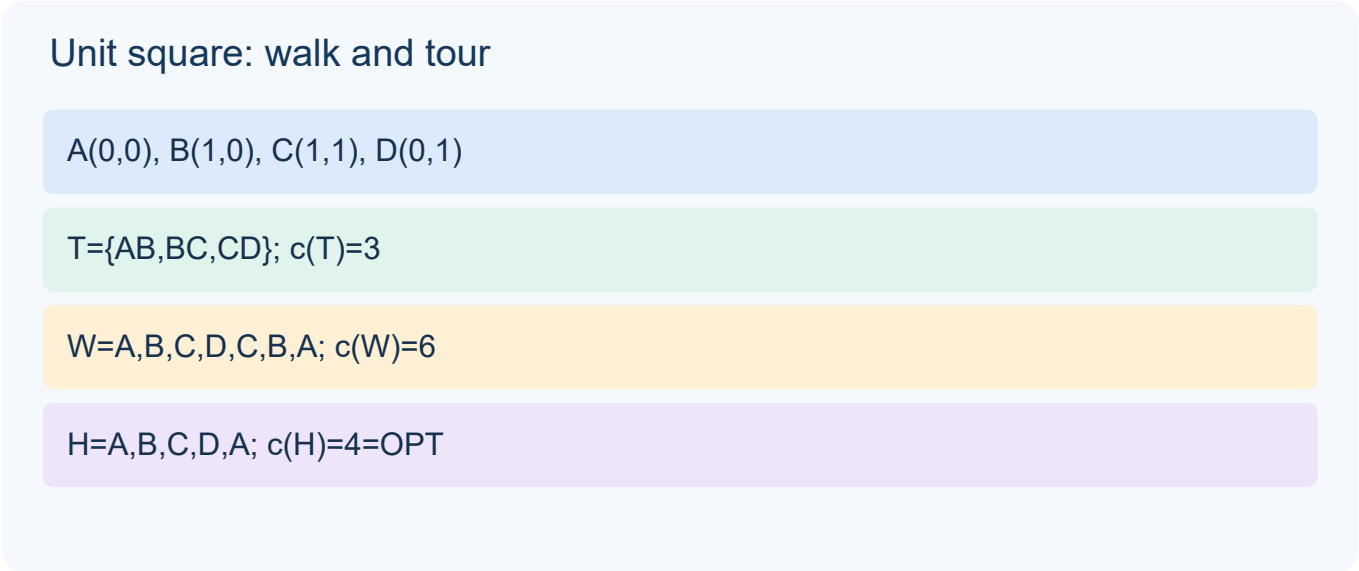
M is maximal because no edge can be added at termination; it need not be maximum. On a-b-c-d, {bc} is maximal of size 1, while {ab,cd} is maximum of size 2. Taking only one arbitrary endpoint per matching edge can leave other edges uncovered.

A11-A04 From walk to tour

For the unit square and MST={AB,BC,CD}, list the full walk and shortcut tour, compute costs, prove OPT, and identify the assumptions in the factor-2 proof.

Use Euclidean distances on A=(0,0), B=(1,0), C=(1,1), D=(0,1). An MST uses AB, BC, CD and costs 3.

The full walk A,B,C,D,C,B,A costs 6. Keeping first visits and closing gives A,B,C,D,A of cost 4. Every Hamiltonian cycle has four edges of length at least 1, so OPT≥4; the displayed tour attains 4, proving OPT=4.



Delete any edge from an optimal tour H^* to obtain a spanning tree. Nonnegative costs prevent this deletion from increasing cost. MST optimality gives $c(T) \le c(H^*)$.

W traverses each tree edge twice, so $c(W)=2c(T)$. Between consecutive retained vertices, let the original walk segment be $u=v_0,v_1,\ldots,v_k=w$. Repeated triangle inequalities give $c(u,w) \le \sum_j c(v_{j-1},v_j)$. These segments partition W, so $c(H) \le c(W)$.

Therefore $c(H) \le c(W)=2c(T) \le 2\mathrm{OPT}$. Dense Prim takes $O(n^2)$; walking and retaining first visits take $O(n)$, for $O(n^2)$ overall.

A11-A05 A nonmetric counterexample

Perimeter edges cost 1 and AC=BD=M>4. Use MST AB,AD,BC rooted at A, visiting D first. Prove the preorder ratio is unbounded.

Let AB=BC=CD=DA=1 and AC=BD=M>4 in a complete four-vertex graph. An MST can use AB, AD, BC rooted at A. Visiting D before B gives preorder A,D,B,C and tour cost 2+2M. The perimeter tour is optimal with cost 4.

The full walk A,D,A,B,C,B,A costs 6. Replacing D→A→B by D→B raises cost from 2 to M; replacing C→B→A by C→A also raises cost. The ratio (2+2M)/4 is unbounded. At M=10 it is 22/4=5.5.

This defeats the double-tree guarantee. The stronger impossibility of every constant approximation unless P=NP requires a separate gap reduction.

A11-A06 The full set-cover charging proof

Use A={1,2,3,4}, B={1,2,5}, C={3,4,6}, with alphabetical ties. Find greedy, OPT, and charges. Prove the local harmonic bound and global H_d guarantee.

Given universe U and family F with $\cup F=U$, choose as few sets as possible to cover U. Repeatedly select S maximizing $|S\cap R|$, where R is uncovered, until $R=\varnothing$. Break ties alphabetically.

Let U={1,…,6}, A={1,2,3,4}, B={1,2,5}, C={3,4,6}. Greedy selects A for four new elements, B for element 5, and C for element 6. It uses 3 sets. $B\cup C=U$ and no set covers U alone, so OPT=2.

Charges at first coverage

A={1,2,3,4}; B={1,2,5}; C={3,4,6}

A → 1,2,3,4: each 1/4

B → 5: 1; C → 6: 1

Total=3; OPT=2; H4=25/12

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Charging proof

If a round covers k new elements, charge each $c_x=1/k$. Each element is charged once, so the greedy set count is $|G|=\Sigma_x c_x$.

In the example, $c_1=c_2=c_3=c_4=1/4$ and $c_5=c_6=1$. The total is $4\times 1/4+1+1=3$. Set B carries charge 3/2, as does C.

For any optimal cover F*, each element occurs in at least one chosen set. Nonnegative charges imply $\sum_x c_x \le \sum_{S\in F^*}\sum_{x\in S} c_x$. Counting overlap twice only increases the right side. It remains to bound the charge inside any one set.

Fix any $S\in F$ with $s=|S|$. Order its elements $x_1,\ldots,x_s$ by first coverage, breaking ties within a round arbitrarily.

At the start of the round covering x_j, at least $s-j+1$ elements of S remain uncovered. Earlier elements from the same round only make this lower bound weaker. The greedy set covers $k\ge|S\cap R|\ge s-j+1$ new elements, since otherwise S would be a better choice.

Thus $c_{x_j} = 1/k \le 1/(s-j+1)$. Summing gives $\sum_{x\in S} c_x \le \sum_{j=1}^{s} 1/(s-j+1) = H_s$. This applies to every candidate set, including the sets in the optimal cover.

If U=∅, greedy and optimal covers are empty and OPT=0. The cost inequality holds; do not evaluate 0/0, a maximum over an empty family, or ln 0. For U≠∅, set H_0=0 and $d = \max_{S\in F} |S| \ge 1$. Combining local charges with the optimal cover gives

$|G| = \sum_x c_x \le \sum_{S\in F^*} \sum_{x\in S} c_x \le \sum_{S\in F^*} H_{|S|} \le H_d |F^*|$.

Since $H_d = 1 + \sum_{j=2}^{d} 1/j \le 1 + \int_1^d dx/x = 1 + \ln d \le 1 + \ln |U|$, greedy is an H_d approximation. Here d=4 and H_4=25/12; the bound is 3≤(25/12)×2=25/6, while the actual ratio is 3/2.

A simple rescan takes O(mN) per round and at most N rounds, hence $O(mN^2)$ for m=|F| and N=|U|. This proves polynomial time. Do not drop the additive 1 in the logarithmic bound.

A11-A07 Trace every trimmed list

For x=(104,102,201,101), t=308, ε=0.4, show merged and trimmed lists, deletion reasons, returned subset, OPT, and actual ratio.

Take x=(104,102,201,101), t=308, ε=0.4, n=4, δ=0.05. Use exact comparisons to avoid floating-point boundary errors.

i	Merged list	After trimming and removing >308
1	0,104	0,104
2	0,102,104,206	0,102,206
3	0,102,201,206,303,407	0,102,201,303
4	0,101,102,201,203,302,303,404	0,101,201,302

Round 2 removes 104≤102×1.05=107.1. Round 3 removes 206≤201×1.05=211.05. Round 4 removes 303≤302×1.05=317.1. The answer is 302=201+101; OPT=307=104+102+101. Thus 307/302≈1.01656≤1.4.

Round 2 trims 104. Round 3 trims 206 and removes 407 by capacity. Round 4 trims 102,203,303 and removes 404 by capacity.

A11-A08 Two obligations of an FPTAS proof

Prove the error invariant and accumulated product bound. Prove polynomial bit complexity in input length and 1/ε. Explain why O(nt) is insufficient.

Claim that each $y \in P_i$ with $y \le t$ has $z \in L_i$ satisfying $y/(1+\delta)^i \le z \le y$. The base case $i=0$ contains only 0.

If y omits x_i, use its previous representative q. If $y=p+x_i$, a representative q of p satisfies $q+x_i \ge p/(1+\delta)^{i-1}+x_i \ge (p+x_i)/(1+\delta)^{i-1}$ because $x_i \ge 0$. In both cases the merged list contains $w \in [y/(1+\delta)^{i-1}, y]$.

Trimming retains $z \in [w/(1+\delta), w]$, proving the claim. Since $z \le y \le t$, capacity deletion does not remove z. Applying the claim to OPT yields $A \ge OPT/(1+\delta)^n$.

Using $\ln(1+\delta) \le \delta$ gives $(1+\delta)^n \le e^{\varepsilon/2} \le 1+\varepsilon$. For the last inequality, $f(\varepsilon)=\ln(1+\varepsilon)-\varepsilon/2$ has $f(0)=0$ and $f'(\varepsilon)=1/(1+\varepsilon)-1/2 \ge 0$ for $0 \le \varepsilon \le 1$. Thus the multiplicative guarantee $A \ge OPT/(1+\varepsilon)$ holds in all cases. If $OPT=0$, feasibility gives $A=0$ and no ratio is taken. If $OPT>0$, the bound implies $A>0$, so division yields $OPT/A \le 1+\varepsilon$.

After trimming, consecutive positive values have ratio $>1+\delta$ and lie in $[1,t]$. Therefore $|L_i| \le 2+\ln t/\ln(1+\delta)$. Since $\ln(1+\delta) \ge \delta/(1+\delta)$ and $\delta=\varepsilon/(2n)<1/2$, $|L_i| \le 2+3n \ln t/\varepsilon$.

Write $K=O(1+n \log t/\varepsilon)$. Linear merging of sorted lists takes $O(K)$ arithmetic operations per round, or $O(nK)$ overall. General sorting adds $O(\log K)$ but remains polynomial.

Binary input length B includes every x_i, t, and the rational encoding of ε. Retained integers use $O(\log t)$ bits; merged values are at most $t+\max x_i$. Rational threshold comparisons use cross multiplication, so bit complexity is polynomial in B and $1/\varepsilon$. Do not treat ε as a free infinite-precision real.

A PTAS needs polynomial time in B for fixed ε, such as $B^{1/\varepsilon}$; an FPTAS also needs polynomial dependence on $1/\varepsilon$. Trimming meets the latter requirement.

A11-Y01

Initial prices are A=2, B=1, C=1. Break the tie toward B, charging elements 1 and 2 one unit each. With 3 and 4 left, C has price 1 versus A's 4. Select C and charge each remaining element 1. Cost is 4. A costs at least 8, and a cover without A needs B and C, so OPT=4. When $M > 4$, the most-new-elements rule selects A for M while OPT remains 4; its ratio $M/4$ is unbounded.

A11-Y02

Each clause is satisfied with probability $7/8$, giving initial expectation $7/4$. Given $a = 0$, the first clause becomes $b \vee c$ and the second is true, yielding $3/4 + 1 = 7/4$. By symmetry $a = 1$ also yields $7/4$. After $a = 0$, choosing $b = 0$ yields $1/2 + 1 = 3/2$, while $b = 1$ makes both true and gives 2. Assignment $(a, b, c) = (0, 1, 0)$ satisfies both.

A11-Y03

Selecting b alone covers both edges at cost 3. The LP constraints imply $x_a + x_c \geq 2 - 2x_b$, so $2x_a + 3x_b + 2x_c \geq 4 - x_b \geq 3$. The feasible solution $(0, 1, 0)$ reaches 3, establishing $LP^* = 3$. Threshold rounding selects b at cost 3. Integer solutions form a subset of LP solutions, so OPT is also 3.

A11-Y04

Each element's three variables sum to 1, so this solution costs 2. Adding the two element constraints lower-bounds every LP solution by 2, hence $LP^* = 2$. Here $N = 2$, $k = 3$, and each $p = \min(1, 3/3) = 1$. All six sets are selected for cost 6 and cover both elements; $6 \leq 2kLP^* = 12$. This example always covers, but other instances can have $p < 1$ and require the failure analysis and actual coverage check.

A11-Y05

$M = Rn + 1 = 9$ and threshold $Rn = 8$. Starting at 1 and identifying reverse tours, the costs are 1-2-3-4-1: $1 + 1 + 1 + 9 = 12$; 1-2-4-3-1: $1 + 9 + 1 + 9 = 20$; 1-3-2-4-1: $9 + 1 + 9 + 9 = 28$. All exceed 8, consistent with the path having no HC. Conversely, if the original graph had an HC, the approximation output would cost at most 8.

A11-C01 Solution

The three star edges each cost 1. Every spanning tree has three edges costing at least 1, so the star is an MST of cost 3. The center has degree 3 and each leaf degree 1, making all four vertices odd. Every perfect matching pairs the center with a leaf at cost 1 and pairs the other leaves at cost 2; the minimum cost is 3. Choose $M = \{oa, bc\}$. Edge oa has one copy in the tree and one in the matching.

An Euler closed walk is $o \to a \to o \to b \to c \to o$, costing $1 + 1 + 1 + 2 + 1 = 6$. Skipping the intermediate o gives $o \to a \to b \to c \to o$, costing $1 + 2 + 2 + 1 = 6$. Every four-vertex tour has two center edges and two leaf-to-leaf edges, so each costs 6 and OPT=6. Removing one copy of oa makes the degrees of o, a odd and no longer guarantees an Euler closed walk.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd edition, chapters 34 and 35. MIT 6.046J Fall 2004, Lecture 24, approximation algorithms. Original handwritten algorithms notes, complexity and approximation pages.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study